

第30章 电容触摸屏—触摸画板

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

关于开发板配套的结构尺寸参数可查阅《4.5 电容触摸屏结构尺寸》，触摸面板配套的触摸控制芯片可查阅《GT5688 Datasheet》及《gt91x 编程指南》配套资料获知。。

在前面我们学习了如何使用 FSMC 外设控制液晶屏并用它显示各种图形及文字，利用液晶屏，STM32 的系统具有了高级信息输出功能，然而我们还希望有用户友好的输入设备，触摸屏是不二之选，目前大部分电子设备都使用触摸屏配合液晶显示器组成人机交互系统。

30.1 触摸屏简介

触摸屏又称触控面板，它是一种把触摸位置转化成坐标数据的输入设备，根据触摸屏的检测原理，主要分为电阻式触摸屏和电容式触摸屏。相对来说，电阻屏造价便宜，能适应较恶劣的环境，但它只支持单点触控(一次只能检测面板上的一个触摸位置)，触摸时需要一定的压力，使用久了容易造成表面磨损，影响寿命；而电容屏具有支持多点触控、检测精度高的特点，电容屏通过与导电物体产生的电容效应来检测触摸动作，只能感应导电物体的触摸，湿度较大或屏幕表面有水珠时会影响电容屏的检测效果。



图 30-1 单电阻屏、电阻液晶屏（带触摸控制芯片）



图 30-2 单电容屏、电容液晶屏(带触摸控制芯片)

图 30-1 和图 30-2 分别是带电阻触摸屏及电容触摸屏的两种屏幕，从外观上并没有明显的区别，区分电阻屏与电容屏最直接的方法就是使用绝缘物体点击屏幕，因为电阻屏通

过压力能正常检测触摸动作，而该绝缘物体无法影响电容屏所检测的信号，因而无法检测到触摸动作。目前电容式触摸屏被大部分应用在智能手机、平板电脑等电子设备中，而在汽车导航、工控机等设备中电阻式触摸屏仍占主流。

30.1.1 电阻式触摸屏检测原理

电阻式的触摸屏结构见图 30-3。它主要由表面硬涂层、两个 ITO 层、间隔点以及玻璃底层构成，这些结构层都是透明的，整个触摸屏覆盖在液晶面板上，透过触摸屏可看到液晶面板。表面涂层起到保护作用，玻璃底层起承载的作用，而两个 ITO 层是触摸屏的关键结构，它们是涂有铟锡金属氧化物的导电层。两个 ITO 层之间使用间隔点使两层分开，当触摸屏表面受到压力时，表面弯曲使得上层 ITO 与下层 ITO 接触，在触点处连通电路。



图 30-3 电阻式触摸屏结构

两个 ITO 涂层的两端分别引出 X-、X+、Y-、Y+ 四个电极，见图 30-4，这是电阻屏最常见的四线结构，通过这些电极，外部电路向这两个涂层可以施加匀强电场或检测电压。

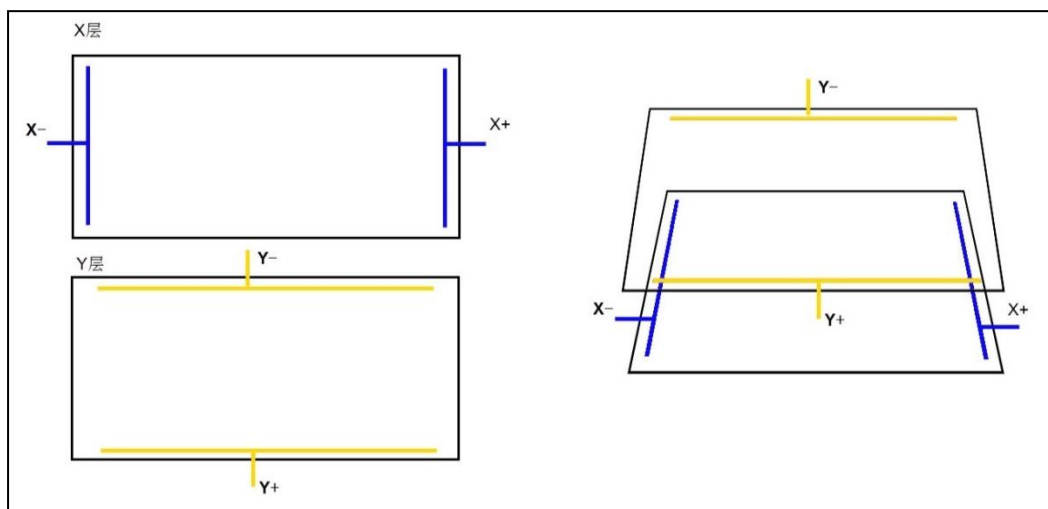


图 30-4 XY 的 ITO 层结构

当触摸屏被按下时，两个 ITO 层相互接触，从触点处把 ITO 层分为两个电阻，且由于 ITO 层均匀导电，两个电阻的大小与触点离两电极的距离成比例关系，利用这个特性，可通过以下过程来检测坐标，这也正是电阻触摸屏名称的由来，见图 30-5。

- 计算 X 坐标时，在 X+ 电极施加驱动电压 V_{ref} ，X- 极接地，所以 X+ 与 X- 处形成了匀强电场，而触点处的电压通过 Y+ 电极采集得到，由于 ITO 层均匀导电，触点电压与 V_{ref} 之比等于触点 X 坐标与屏宽度之比，从而：

$$x = \frac{V_{Y+}}{V_{ref}} \times Width$$

- 计算 Y 坐标时，在 Y+ 电极施加驱动电压 V_{ref} ，Y- 极接地，所以 Y+ 与 Y- 处形成了匀强电场，而触点处的电压通过 X+ 电极采集得到，由于 ITO 层均匀导电，触点电压与 V_{ref} 之比等于触点 Y 坐标与屏高度之比，从而：

$$y = \frac{V_{X+}}{V_{ref}} \times Height$$

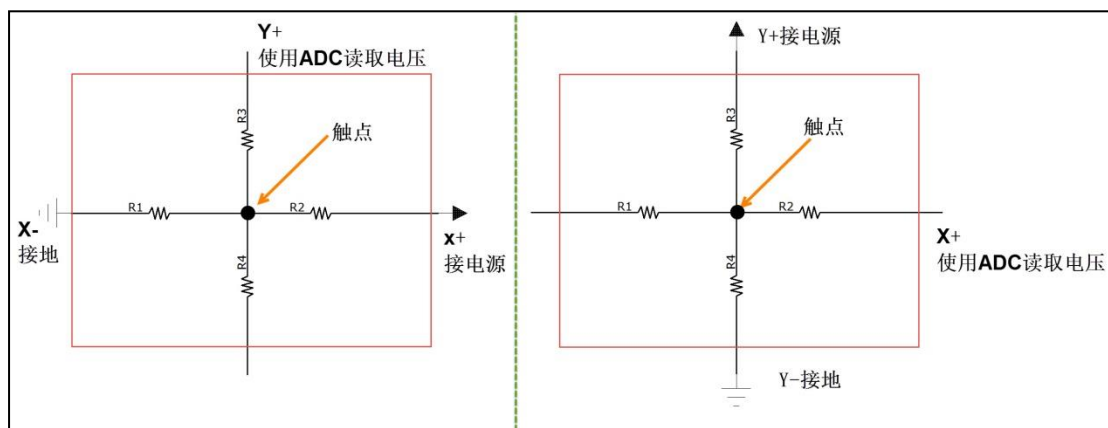


图 30-5 触摸检测等效电路

为了方便检测触摸的坐标，一些芯片厂商制作了电阻屏专用的控制芯片，控制上述采集过程、采集电压，外部微控制器直接与触摸控制芯片通讯直接获得触点的电压或坐标。如图 30-1 中我们生产的这款 3.2 寸电阻触摸屏就是采用 XPT2046 芯片作为触摸控制芯片，XPT2046 芯片控制 4 线电阻触摸屏，STM32 与 XPT2046 采用 SPI 通讯获取采集得的电压，然后转换成坐标。

30.1.2 电容式触摸屏检测原理

与电阻式触摸屏不同，电容式触摸屏不需要通过压力使触点变形，再通过触点处电压值来检测坐标，它的基本原理和前面定时器章节中介绍的电容按键类似，都是利用充电时间来检测电容大小，从而通过检测出电容值的变化来获知触摸信号。见图 30-6，电容屏的最上层是玻璃(不会像电阻屏那样形变)，核心层部分也是由 ITO 材料构成的，这些导电材料在屏幕里构成了人眼看不见的静电网，静电网由多行 X 轴电极和多列 Y 轴电极构成，两个电极之间会形成电容。触摸屏工作时，X 轴电极发出 AC 交流信号，而交流信号能穿过电容，即通过 Y 轴能感应出该信号，当交流电穿越时电容会有充放电过程，检测该充电时间可获知电容量。若手指触摸屏幕，会影响触摸点附近两个电极之间的耦合，从而改变两个

电极之间的电容量，若检测到某电容的电容量发生了改变，即可获知该电容处有触摸动作（这就是为什么它被称为电容式触摸屏以及绝缘体触摸没有反应的原因）。

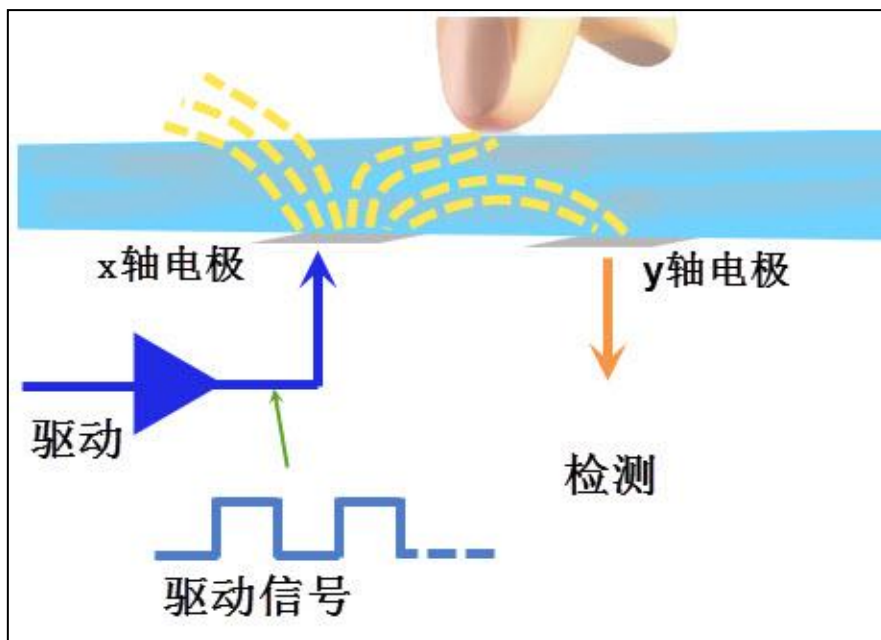


图 30-6 电容触摸屏基本原理

电容屏 ITO 层的结构见图 30-7，这是比较常见的形式，电极由多个菱形导体组成，生产时使用蚀刻工艺在 ITO 层生成这样的结构。

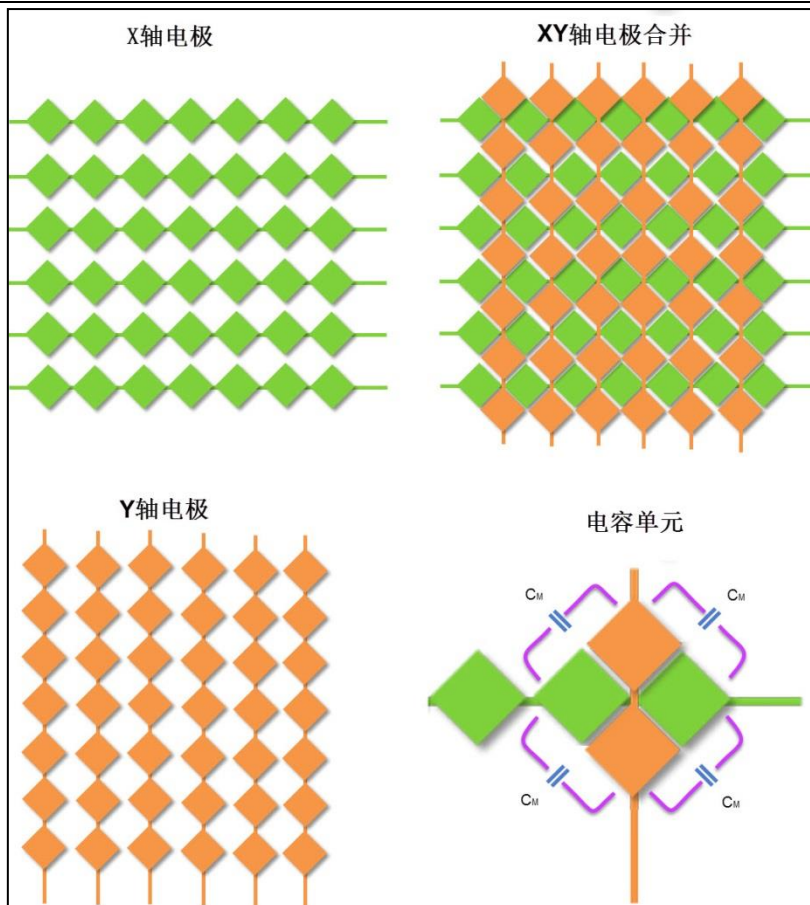


图 30-7 电容触摸屏的 ITO 层结构

X 轴电极与 Y 轴电极在交叉处形成电容，即这两组电极构成了电容的两极，这样的结构覆盖了整个电容屏，每个电容单元在触摸屏中都有其特定的物理位置，即电容的位置就是它在触摸屏的 XY 坐标。检测触摸的坐标时，第 1 条 X 轴的电极发出激励信号，而所有 Y 轴的电极同时接收信号，通过检测充电时间可检测出各个 Y 轴与第 1 条 X 轴相交的各个互电容的大小，各个 X 轴依次发出激励信号，重复上述步骤，即可得到整个触摸屏二维平面的所有电容大小。当手指接近时，会导致局部电容改变，根据得到的触摸屏电容量变化的二维数据表，可以得知每个触摸点的坐标，因此电容触摸屏支持多点触控。

其实电容触摸屏可看作是多个电容按键组合而成，就像机械按键中独立按键和矩阵按键的关系一样，甚至电容触摸屏的坐标扫描方式与矩阵按键都是很相似的。

30.2 电容触摸屏控制芯片

相对来说，电容屏的坐标检测比电阻屏的要复杂，因而它也有专用芯片用于检测过程，下面我们以本章重点讲述的电容屏使用的触控芯片 GT5688 为例进行讲解，关于它的详细说明可从《gt91x 编程指南》和《GT5688 Datasheet》文档了解。

30.2.1 GT5688 芯片的引脚

GT5688 芯片的外观可以图 30-2 中找到，其内部结构框图见图 30-8。

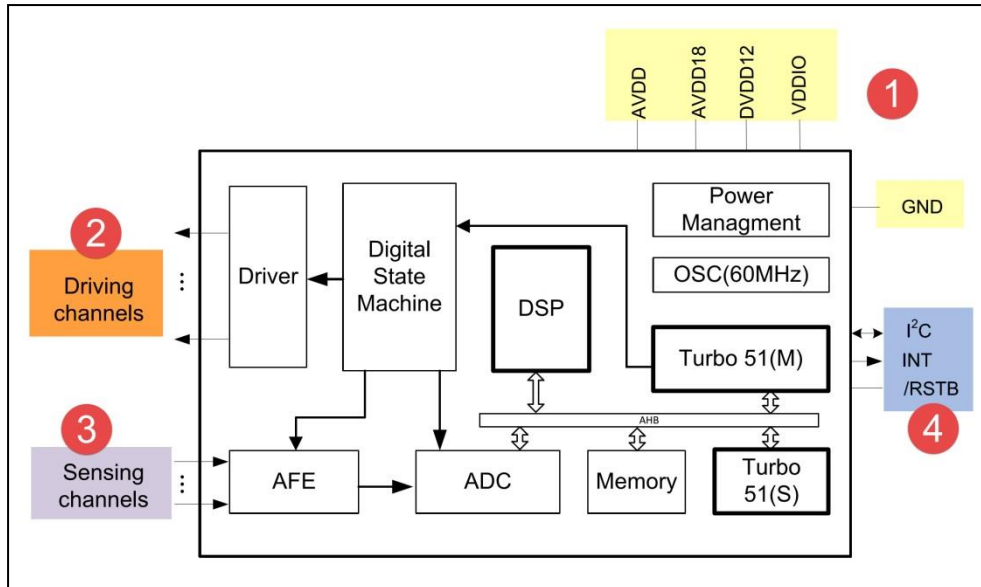


图 30-8 GT5688 结构框图

该芯片对外引出的信号线介绍如下：

表 30-1 GT5688 信号线说明

信号线	说明
AVDD、AVDD18、DVDD12、VDDIO、GND	电源和地
Driving channels	激励信号输出的引脚，一共有 0-25 个引脚，它连接到电容屏 ITO 层引出的各个激励信号轴
Sensing channels	信号检测引脚，一共有 0-13 个引脚，它连接到电容屏 ITO 层引出的各个电容量检测信号轴
I2C	I2C 通信信号线，包含 SCL 与 SDA，外部控制器通过它与 GT5688 芯片通讯，配置 GT5688 的工作方式或获取坐标信号
INT	中断信号，GT5688 芯片通过它告诉外部控制器有新的触摸事件
/RSTB	复位引脚，用于复位 GT5688 芯片；在上电时还与 INT 引脚配合设置 IIC 通讯的设备地址

若您把电容触摸屏与液晶面板分离开来，在触摸面板的背面，可看到它的边框有一些电路走线，它们就是触摸屏 ITO 层引出的 XY 轴信号线，这些信号线分别引出到 GT5688 芯片的 Driving channels 及 Sensing channels 引脚中。也正是因为触摸屏有这些信号线的存在，所以手机厂商追求的屏幕无边框是比较难做到的。

30.2.2 上电时序与 I2C 设备地址

GT5688 触控芯片有两个备选的 I2C 通讯地址，这是由芯片的上电时序设定的，见图 30-9。上电时序有 Reset 引脚和 INT 引脚生成，若 Reset 引脚从低电平转变到高电平期间，INT 引脚为高电平的时候，触控芯片使用的 I2C 设备地址为 0x28/0x29(8 位写、读地址)，7 位地址为 0x14；若 Reset 引脚从低电平转变到高电平期间，INT 引脚一直为低电平，则触控芯片使用的 I2C 设备地址为 0xBA/0xBB(8 位写、读地址)，7 位地址为 0x5D。

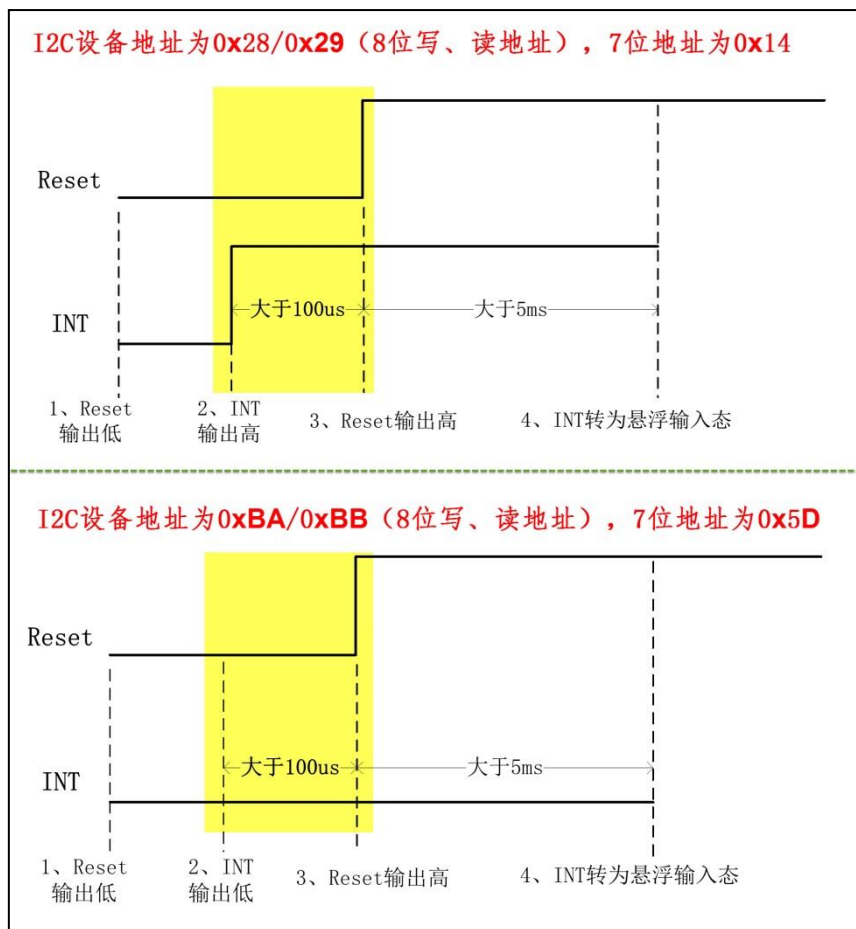


图 30-9 GT5688 的上电时序及 I2C 设备地址

30.2.3 寄存器配置

上电复位后，GT5688 芯片需要通过外部主控芯片加载寄存器配置，设定它的工作模式，这些配置通过 I2C 信号线传输到 GT5688，它的配置寄存器地址都由两个字节来表示，这些寄存器的地址从 0x8047- 0x8135，一般来说，我们实际配置的时候会按照 GT5688 生产厂商给的默认配置来控制芯片，仅修改部分关键寄存器，其中部分寄存器说明见图 30-10。

（注意，由于芯片厂商并没有发布 GT5688 的寄存器列表说明文档，此处使用旧型号芯片 GT9157 的寄存器表截图来进行说明，这两个型号大部分兼容）

寄存器	Config Data	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
0x8047	Config_Version	配置文件的版本号(新下发的配置版本号大于原版本,或等于原版本号但配置内容有变化时保存,版本号版本正常范围: 'A'~'Z',发送 0x00 则将版本号初始化为'A')							
0x8048	X Output Max (Low Byte)	x 坐标输出最大值							
0x8049	X Output Max (High Byte)								
0x804A	Y Output Max (Low Byte)	y 坐标输出最大值							
0x804B	Y Output Max (High Byte)								
0x804C	Touch Number	Reserved				输出触点个数上限: 1~10			
0x804D	Module_Switch1	Stylus_priority (预定义)		Stretch_rank		X2Y (X,Y 坐标交 换)	Sito (软件 降噪)	INT 触发方式 00: 上升沿触发 01: 下降沿触发 02: 低电平查询 03: 高电平查询	
0x804E-0x80FE寄存器省略									
0x80FF	Config_Chksum	配置信息校验(0x8047 到 0x80FE 之字节和的补码)							
0x8100	Config_Fresh	配置已更新标记(由主控写入标记)							

图 30-10 部分寄存器配置说明

这些寄存器介绍如下:

(1) 配置版本寄存器

0x8047 配置版本寄存器,它包含有配置文件的版本号,若新写入的版本号比原版本大,或者版本号相等,但配置不一样时,才会更新配置文件到寄存器中。其中配置文件是指记录了寄存器 0x8048-0x80FE 控制参数的一系列数据。

为了保证每次都更新配置,我们一般把配置版本寄存器设置为“0x00”,这样版本号会默认初始化为‘A’,这样每次我们修改其它寄存器配置的时候,都会写入到 GT5688 中。

(2) X、Y 分辨率

0x8048-0x804B 寄存器用于配置触控芯片输出的 XY 坐标的最大值,为了方便使用,我们把它配置得跟液晶面板的分辨率一致,这样就能使触控芯片输出的坐标一一对应到液晶面板的每一个像素点了。

(3) 触点个数

0x804C 触点个数寄存器用于配置它最多可输出多少个同时按下的触点坐标,这个极限值跟触摸屏面板有关,如我们本章实验使用的触摸面板最多支持 5 点触控。

(4) 模式切换

0x804D 模式切换寄存器中的 X2Y 位可以用于交换 XY 坐标轴;而 INT 触发方式位可以配置不同的触发方式,当有触摸信号时,INT 引脚会根据这里的配置给出触发信号。

(5) 配置校验

0x80FF 配置校验寄存器用于写入前面 0x8047-0x80FE 寄存器控制参数字节之和的补码，GT5688 收到前面的寄存器配置时，会利用这个数据进行校验，若不匹配，就不会更新寄存器配置。

(6) 配置更新

0x8100 配置更新寄存器用于控制 GT5688 进行更新，传输了前面的寄存器配置并校验通过后，对这个寄存器写 1，GT5688 会更新配置。

30.2.4 读取坐标信息

坐标寄存器

上述寄存器主要是由外部主控芯片给 GT5688 写入配置的，而它则使用图 30-11 中的寄存器向主控器反馈信息。

Addr	Access	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
0x8140	R	Product ID (Lowest Byte, ASCII 码)							
0x8141	R	Product ID (Third Byte, ASCII 码)							
0x8142	R	1	Product ID (Second Byte, ASCII 码)						
0x8143	R		Product ID (Highest Byte, ASCII 码)						
0x8144	R		Firmware version (16 进制数 LowByte)						
0x8145	R	Firmware version (16 进制数 HighByte)							
0x8146	R	x coordinate resolution (low byte)							
0x8147	R	2	x coordinate resolution (high byte)						
0x8148	R		y coordinate resolution (low byte)						
0x8149	R		y coordinate resolution (high byte)						
0x814A	R	Vendor_id (当前模组选项信息)							
0x814B	R	Reserved							
0x814C	R	3	Reserved						
0x814D	R		Reserved						
0x814E	R/W	buffer status	large detect	Proximity Valid	HaveKey	number of touch points			
0x814F	R	track id							
0x8150	R	point 1 x coordinate (low byte)							
0x8151	R	point 1 x coordinate (high byte)							
0x8152	R	4	point 1 y coordinate (low byte)						
0x8153	R		point 1 y coordinate (high byte)						
0x8154	R		Point 1 size (low byte)						
0x8155	R	point 1 size (high byte)							
0x8156	R	Reserved							
0x8157	R	track id							
0x8158	R	point 2 x coordinate (low byte)							
0x8159	R	point 2 x coordinate (high byte)							
0x815A	R	5	point 2 y coordinate (low byte)						
0x815B	R		point 2 y coordinate (high byte)						
0x815C	R		point 2 size (low byte)						
0x815D	R	point 2 size (high byte)							
0x815E	R	Reserved							

省略track id3-track ic10的寄存器

图 30-11 坐标信息寄存器

(1) 产品 ID 及版本

0x8140-0x8143 寄存器存储的是产品 ID，上电后我们可以利用 I2C 读取这些寄存器的值来判断 I2C 是否正常通讯，这些寄存器中包含有“5688”字样；而 0x8144-0x8145 则保存有固件版本号，不同版本可能不同。

(2) X/Y 分辨率

0x8146-0x8149 寄存器存储了控制触摸屏的分辨率，它们的值与我们前面在配置寄存器写入的 XY 控制参数一致。所以我们可以通过读取这两个寄存器的值来确认配置参数是否正确写入。

(3) 状态寄存器

0x814E 地址的是状态寄存器，它的 Buffer status 位存储了坐标状态，当它为 1 时，表示新的坐标数据已准备好，可以读取，0 表示未就绪，数据无效，外部控制器读取完坐标后，须对这个寄存器位写 0。number of touch points 位表示当前有多少个触点。其余数据位我们不关心。

(4) 坐标数据

从地址 0x814F-0x8156 的是触摸点 1 的坐标数据，从 0x8157-0x815E 的是触摸点 2 的坐标数据，依次还有存储 3-10 触摸点坐标数据的寄存器。读取这些坐标信息时，我们通过它们的 track id 来区分笔迹，多次读取坐标数据时，同一个 track id 号里的数据属于同一个连续的笔划轨迹。

读坐标流程

上电、配置完寄存器后，GT5688 就会开监测触摸屏，若我们前面的配置使 INT 采用中断上升沿报告触摸信号的方式，整个读取坐标信息的过程如下：

- (1) 待机时 INT 引脚输出低电平；
- (2) 有坐标更新时，INT 引脚输出上升沿；
- (3) INT 输出上升沿后，INT 脚会保持高直到下一个周期（该周期可由配置 Refresh_Rate 决定）。外部主控器在检测到 INT 的信号后，先读取状态寄存器(0x814E)中的 number of touch points 位获当前有多少个触摸点，然后读取各个点的坐标数据，读取完后将 buffer status 位写为 0。外部主控器的这些读取过程要在一周期内完成，该周期由 0x8056 地址的 Refresh_Rate 寄存器配置；
- (4) 上一步骤中 INT 输出上升沿后，若主控未在一个周期内读走坐标，下次 GT5688 即使检测到坐标更新会再输出一个 INT 脉冲但不更新坐标；
- (5) 若外部主控一直未读走坐标，则 GT5688 会一直输出 INT 脉冲。

30.3 电容触摸屏—触摸画板实验

本小节讲解如何驱动电容触摸屏，并利用触摸屏制作一个简易的触摸画板应用。

学习本小节内容时，请打开配套的“电容触摸屏—触摸画板”工程配合阅读。

30.3.1 硬件设计



图 30-12 液晶屏实物图

本实验使用的液晶电容屏实物见图 28-18，屏幕背面的 PCB 电路对应图 28-19、图 28-20 中的原理图，分别是触摸屏接口及排针接口。

我们这个触摸屏出厂时就与 GT5688 芯片通过柔性电路板连接在一起了，柔性电路板从 GT5688 芯片引出 VCC、GND、SCL、SDA、RSTN 及 INT 引脚，再通过 FPC 座子引出到屏幕的 PCB 电路板中，PCB 电路板加了部分电路，如 I2C 的上拉电阻，然后把这些引脚引出到屏幕下方的排针处，方便整个屏幕与外部器件相连。

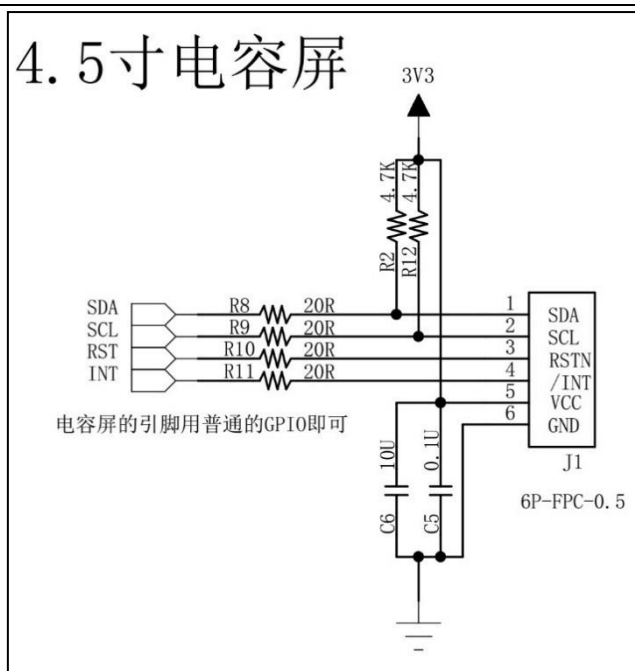


图 30-13 电容屏接口

以上是我们 STM32F407 实验板使用的 4.5 寸屏原理图，它通过屏幕上的排针接入到实验板的液晶排母接口，与 STM32 芯片的引脚相连，连接见图 30-14。

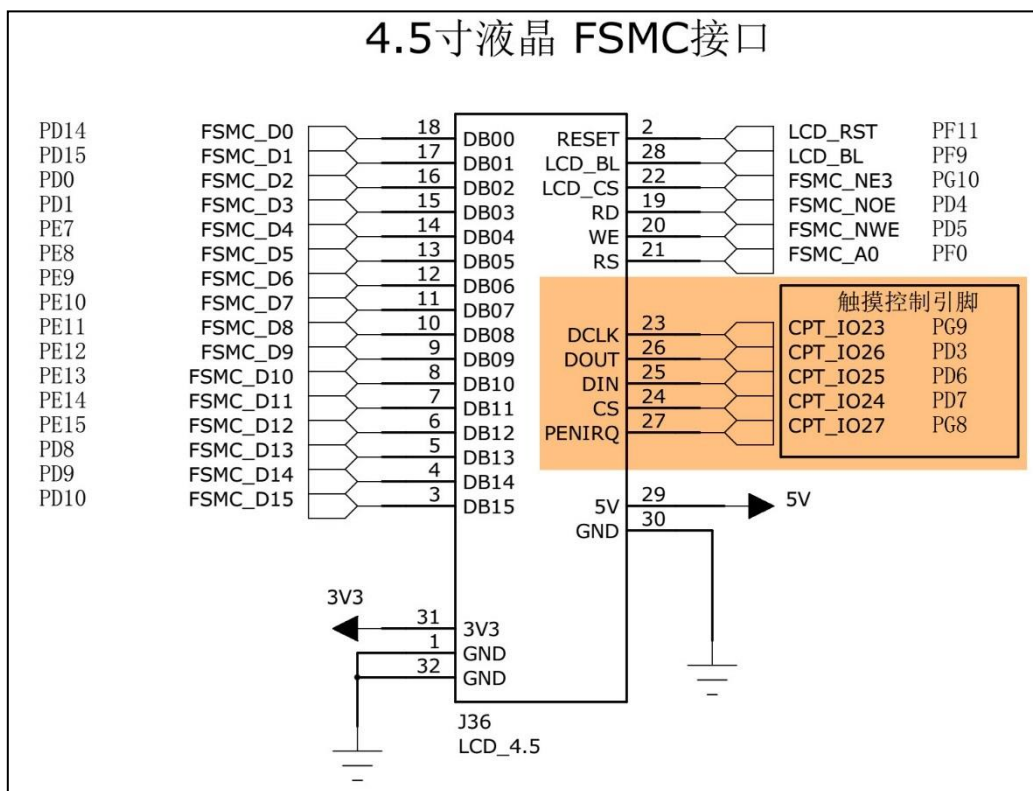


图 30-14 屏幕与实验板的引脚连接

图 30-14 中 23~27 号引脚即电容触摸屏相关的控制引脚。在本硬件设计中，没有使用 STM32 的硬件 I2C 引脚，只使用普通的 GPIO 与触摸芯片的 SCL、SDA 连接，在程序上使用软件模拟 I2C 与触摸芯片进行通讯。

以上原理图可查阅《LCD4.5 寸液晶原理图》及《野火 F407 开发板底板原理图》文档获知，若您使用的液晶屏或实验板不一样，请根据实际连接的引脚修改程序。

30.3.2 软件设计

本工程中的 GT5688 芯片驱动主要是从官方提供的 Linux 驱动修改过来的，我们把这部分文件存储到“gt5xx.c”及“gt5xx.h”文件中，而这些驱动的底层 I2C 通讯接口我们存储到了“bsp_i2c_touch.c”及“bsp_i2c_touch.h”文件中，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。在我们提供的资料《GT5663 驱动.zip》压缩包里有官方的原 Linux 驱动，感兴趣的读者可以对比这些文件，了解如何移植驱动。

1. 编程要点

- (1) 分析官方的 gt5xx 驱动，了解需要提供哪些底层接口；
- (2) 编写底层驱动接口；
- (3) 利用 gt5xx 驱动，获取触摸坐标；
- (4) 编写测试程序检验驱动。

2. 代码分析

触摸屏硬件相关宏定义

根据触摸屏与 STM32 芯片的硬件连接，我们把触摸屏硬件相关的配置都以宏的形式定义到“bsp_i2c_touch.h”文件中，见代码清单 25-2。

代码清单 30-1 触摸屏硬件配置相关的宏(bsp_i2c_touch.h 文件)

```
1 /*设定使用的电容屏 IIC 设备地址*/
2 #define GTP_ADDRESS 0xBA
3
4 #define I2CT_FLAG_TIMEOUT ((uint32_t)0x1000)
5 #define I2CT_LONG_TIMEOUT ((uint32_t)(10 * I2CT_FLAG_TIMEOUT))
6
7 /*I2C 引脚*/
8 #define GTP_I2C_SCL_PIN GPIO_Pin_7
9 #define GTP_I2C_SCL_GPIO_PORT GPIOD
10 #define GTP_I2C_SCL_GPIO_CLK RCC_AHB1Periph_GPIOD
11 #define GTP_I2C_SCL_SOURCE GPIO_PinSource7
12
13 #define GTP_I2C_SDA_PIN GPIO_Pin_3
14 #define GTP_I2C_SDA_GPIO_PORT GPIOD
15 #define GTP_I2C_SDA_GPIO_CLK RCC_AHB1Periph_GPIOD
16 #define GTP_I2C_SDA_SOURCE GPIO_PinSource3
17
18 /*复位引脚*/
19 #define GTP_RST_GPIO_PORT GPIOD
20 #define GTP_RST_GPIO_CLK RCC_AHB1Periph_GPIOD
21 #define GTP_RST_GPIO_PIN GPIO_Pin_6
22
23 /*中断引脚*/
24 #define GTP_INT_GPIO_PORT GPIOG
25 #define GTP_INT_GPIO_CLK RCC_AHB1Periph_GPIOG
26 #define GTP_INT_GPIO_PIN GPIO_Pin_8
27 #define GTP_INT_EXTI_PORTSOURCE EXTI_PortSourceGPIOG
28 #define GTP_INT_EXTI_PINSOURCE EXTI_PinSource8
```

```
28 #define GTP_INT_EXTI_LINE          EXTI_Line8
29 #define GTP_INT_EXTI_IRQ           EXTI9_5_IRQn
30 /*中断服务函数*/
31 #define GTP_IRQHandler              EXTI9_5_IRQHandler
```

以上代码根据硬件的连接，把与触摸屏通讯使用的引脚号、引脚源以及复用功能映射都以宏封装起来。在这里还定义了与 GT5688 芯片通讯的 I2C 设备地址，该地址是一个 8 位的写地址，它是由我们的上电时序决定的。

初始化触摸屏控制引脚

利用上面的宏，编写触摸屏控制引脚的初始化函数，见代码清单 25-3。

代码清单 30-2 触摸屏控制引脚的 GPIO 初始化函数(bsp_i2c_touch.c 文件)

```
1
2 /**
3  * @brief  触摸屏 I/O 配置
4  * @param  无
5  * @retval 无
6  */
7 static void I2C_GPIO_Config(void)
8 {
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*使能触摸屏使用的引脚的时钟*/
12     RCC_AHB1PeriphClockCmd(GTP_I2C_SCL_GPIO_CLK|
13                             GTP_I2C_SDA_GPIO_CLK|
14                             GTP_RST_GPIO_CLK|GTP_INT_GPIO_CLK,
15                             ENABLE);
16
17     RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
18
19     /*配置 SCL 引脚 */
20     GPIO_InitStructure.GPIO_Pin = GTP_I2C_SCL_PIN;
21     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
22     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
23     GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
24     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
25     GPIO_Init(GTP_I2C_SCL_GPIO_PORT, &GPIO_InitStructure);
26
27     /*配置 SDA 引脚 */
28     GPIO_InitStructure.GPIO_Pin = GTP_I2C_SDA_PIN;
29     GPIO_Init(GTP_I2C_SDA_GPIO_PORT, &GPIO_InitStructure);
30
31     /*配置 RST 引脚，下拉推挽输出 */
32     GPIO_InitStructure.GPIO_Pin = GTP_RST_GPIO_PIN;
33     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
34     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
35     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
36     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;
37     GPIO_Init(GTP_RST_GPIO_PORT, &GPIO_InitStructure);
38
39     /*配置 INT 引脚，下拉推挽输出，方便初始化 */
40     GPIO_InitStructure.GPIO_Pin = GTP_INT_GPIO_PIN;
41     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
42     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
43     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
44     //设置为下拉，方便初始化
45     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;
46     GPIO_Init(GTP_INT_GPIO_PORT, &GPIO_InitStructure);
47 }
```

以上函数初始化了触摸屏用到的 I2C 信号线，并且把 RST 及 INT 引脚也初始化成了下拉推挽输出模式，以便刚上电的时候输出上电时序，设置触摸屏的 I2C 设备地址。

配置 I2C 的模式

接下来需要配置 I2C 的工作模式，GT9157 芯片使用的是标准 7 位地址模式的 I2C 通讯，所以 I2C 这部分的配置跟我们在 EEPROM 实验中的是一样的，不了解这部分内容的请阅读 EEPROM 章节，见代码清单 25-4。

代码清单 30-3 软件 I2C 相关的部分函数(bsp_i2c_touch.c 文件)

```
1 //软件 IIC 使用的宏
2 #define I2C_SCL_1() GPIO_SetBits(GTP_I2C_SCL_GPIO_PORT, GTP_I2C_SCL_PIN) /* SCL = 1 */
3 #define I2C_SCL_0() GPIO_ResetBits(GTP_I2C_SCL_GPIO_PORT, GTP_I2C_SCL_PIN) /* SCL = 0 */
4
5 #define I2C_SDA_1() GPIO_SetBits(GTP_I2C_SDA_GPIO_PORT, GTP_I2C_SDA_PIN) /* SDA = 1 */
6 #define I2C_SDA_0() GPIO_ResetBits(GTP_I2C_SDA_GPIO_PORT, GTP_I2C_SDA_PIN) /* SDA = 0 */
7
8 #define I2C_SDA_READ() GPIO_ReadInputDataBit(GTP_I2C_SDA_GPIO_PORT, GTP_I2C_SDA_PIN)
9 /* 读 SDA 口线状态 */
10
11 /*
12 *****
13 * 函数名: i2c_Start
14 * 功能说明: CPU 发起 I2C 总线启动信号
15 * 形参: 无
16 * 返回值: 无
17 *****
18 */
19 void i2c_Start(void)
20 {
21     /* 当 SCL 高电平时, SDA 出现一个下跳沿表示 I2C 总线启动信号 */
22     I2C_SDA_1();
23     I2C_SCL_1();
24     i2c_Delay();
25     I2C_SDA_0();
26     i2c_Delay();
27     I2C_SCL_0();
28     i2c_Delay();
29 }
30
31 /*
32 *****
33 * 函数名: i2c_Stop
34 * 功能说明: CPU 发起 I2C 总线停止信号
35 * 形参: 无
36 * 返回值: 无
37 *****
38 */
39 void i2c_Stop(void)
40 {
41     /* 当 SCL 高电平时, SDA 出现一个上跳沿表示 I2C 总线停止信号 */
42     I2C_SDA_0();
43     I2C_SCL_1();
44     i2c_Delay();
45     I2C_SDA_1();
46 }
47
48 /*
49 *****
50 * 函数名: i2c_ReadByte
51 * 功能说明: CPU 从 I2C 总线设备读取 8bit 数据
52 * 形参: 无
53 * 返回值: 读到的数据
54 *****
```

```
55 */
56 uint8_t i2c_ReadByte(void)
57 {
58     uint8_t i;
59     uint8_t value;
60
61     /* 读到第 1 个 bit 为数据的 bit7 */
62     value = 0;
63     for (i = 0; i < 8; i++) {
64         value <<= 1;
65         I2C_SCL_1();
66         i2c_Delay();
67         if (I2C_SDA_READ()) {
68             value++;
69         }
70         I2C_SCL_0();
71         i2c_Delay();
72     }
73     return value;
74 }
```

考虑到硬件 I2C 在实际驱动时存在无法成功发送信号的情况，本工程中关于 I2C 的底层驱动使用软件 I2C。

所谓软件 I2C，即直接控制 STM32 的 IO 引脚的电平，根据 I2C 的时序产生对应的起始信号、结束信号、应答和非应答信号等。上述代码中使用 I2C_SCL_1、I2C_SCL_0 宏控制引脚产生高低电平逻辑，从而产生 I2C 需要的 SCL 时钟信号；I2C_SDA_1、I2C_SDA_0 宏控制 I2C 总线的 SDA 数据线，产生数据输出，当需要读取数据时，使用宏 I2C_SDA_READ。实际上这些宏都是对 GPIO 进行基本的读写，只是把它们按照 I2C 协议的要求来组织，就产生了 I2C 协议的通讯逻辑，实现了 I2C 通讯。（上述代码中省略了部分软件 I2C 的函数，感兴趣的请直接在配套代码中查看）

相对于硬件 I2C，软件 I2C 所有逻辑的产生均由 CPU 直接控制，不过这并不会比前面硬件 I2C 读写 EEPROM 的驱动消耗更多的 CPU 资源，因为在该硬件 I2C 的驱动中，CPU 需要轮询硬件 I2C 的事件标志，并不会省下 CPU 的时间。而软件 I2C 的访问速度可通过调整 i2c_Delay 函数的延时来修改，使用时不会比硬件 I2C 慢。

使用上电时序设置触摸屏的 I2C 地址

在上面配置完成 STM32 的引脚后，就可以开始控制这些引脚对触摸屏进行控制了，为了使用 I2C 通讯，首先要根据 GT5688 芯片的上电时序给它设置 I2C 设备地址，见代码清单 30-4。

代码清单 30-4 使用上电时序设置触摸屏的 I2C 地址(bsp_i2c_touch.c 文件)

```
1 /**
2  * @brief 对 GT91xx 芯片进行复位
3  * @param 无
4  * @retval 无
5  */
6 void I2C_ResetChip(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9
10     /*配置 INT 引脚，下拉推挽输出，方便初始化 */
11     GPIO_InitStructure.GPIO_Pin = GTP_INT_GPIO_PIN;
12     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
13     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
14     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
```



```
15     GPIO_InitStructure.GPIO_PuPd  = GPIO_PuPd_DOWN;    //设置为下拉，方便初始化
16     GPIO_Init(GTP_INT_GPIO_PORT, &GPIO_InitStructure);
17
18     /*初始化 GT5688, rst 为高电平, int 为低电平, 则 gt5688 的设备地址被配置为 0xBA*/
19
20     /*复位为低电平, 为初始化做准备*/
21     GPIO_ResetBits (GTP_RST_GPIO_PORT, GTP_RST_GPIO_PIN);
22     Delay(0xFFFFF);
23
24     /*拉高一段时间, 进行初始化*/
25     GPIO_SetBits (GTP_RST_GPIO_PORT, GTP_RST_GPIO_PIN);
26     Delay(0xFFFFF);
27
28     /*把 INT 引脚设置为浮空输入模式, 以便接收触摸中断信号*/
29     GPIO_InitStructure.GPIO_Pin = GTP_INT_GPIO_PIN;
30     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
31     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
32     GPIO_InitStructure.GPIO_PuPd  = GPIO_PuPd_NOPULL;
33     GPIO_Init(GTP_INT_GPIO_PORT, &GPIO_InitStructure);
34 }
```

这段函数中控制 RST 引脚由低电平改变至高电平，且期间 INT 一直为低电平，这样的上电时序可以控制触控芯片的 I2C 写地址为 0xBA，读地址为 0xBB,即(0xBA|0x01)。输出完上电时序后，把 STM32 的 INT 引脚模式改成浮空输入模式，使它可以接收触控芯片输出的触摸中断信号。接下来我们在 I2C_GTP_IRQEnable 函数中使能 INT 中断，见代码清单 30-5。

代码清单 30-5 使能 INT 中断(bsp_i2c_touch.c 文件)

```
1 /**
2  * @brief  使能触摸屏中断
3  * @param  无
4  * @retval 无
5  */
6 void I2C_GTP_IRQEnable(void)
7 {
8     EXTI_InitTypeDef EXTI_InitStructure;
9     NVIC_InitTypeDef NVIC_InitStructure;
10    GPIO_InitTypeDef GPIO_InitStructure;
11    /*配置 INT 为浮空输入 */
12    GPIO_InitStructure.GPIO_Pin = GTP_INT_GPIO_PIN;
13    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
14    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
15    GPIO_InitStructure.GPIO_PuPd  = GPIO_PuPd_NOPULL;
16    GPIO_Init(GTP_INT_GPIO_PORT, &GPIO_InitStructure);
17
18    /* 连接 EXTI 中断源 到 INT 引脚 */
19    SYSCFG_EXTILineConfig(GTP_INT_EXTI_PORTSOURCE, GTP_INT_EXTI_PINSOURCE);
20
21    /* 选择 EXTI 中断源 */
22    EXTI_InitStructure.EXTI_Line = GTP_INT_EXTI_LINE;
23    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
24    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
25    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
26    EXTI_Init(&EXTI_InitStructure);
27
28    /* 配置中断优先级 */
29    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
30
31    /*使能中断*/
32    NVIC_InitStructure.NVIC_IRQChannel = GTP_INT_EXTI_IRQ;
33    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
34    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
```

```
35     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
36     NVIC_Init(&NVIC_InitStructure);
37
38 }
```

这个 INT 引脚我们配置为上升沿触发，是跟后面写入到触控芯片的配置参数一致的。

初始化封装

利用以上函数，我们把信号引脚及 I2C 设备地址初始化的过程都封装到函数 I2C_Touch_Init 中，见代码清单 30-6。

代码清单 30-6 封装引脚初始化及上电时序(bsp_i2c_touch.c 文件)

```
1  /**
2   * @brief  I2C 外设 (GT5xx) 初始化
3   * @param  无
4   * @retval 无
5   */
6  void I2C_Touch_Init(void)
7  {
8      I2C_GPIO_Config();
9
10     I2C_ResetChip();
11
12     I2C_GTP_IRQEnable();
13 }
```

I2C 基本读写函数

为了与上层“gt5xx.c”驱动文件中的函数对接，本实验中的 I2C 读写函数与 EEPROM 实验中的有稍微不同，见代码清单 30-7。

代码清单 30-7 I2C 基本读写函数(bsp_i2c_touch.c 文件)

```
1
2  #define I2C_DIR_WR  0    /* 写控制 bit */
3  #define I2C_DIR_RD  1    /* 读控制 bit */
4
5  /**
6   * @brief  使用 IIC 读取数据
7   * @param
8   *   @arg ClientAddr:从设备地址
9   *   @arg pBuffer:存放由从机读取的数据的缓冲区指针
10  *   @arg NumByteToRead:读取的数据长度
11  * @retval 无
12  */
13 uint32_t I2C_ReadBytes(uint8_t ClientAddr, uint8_t* pBuffer, uint16_t NumByteToRead)
14 {
15
16     /* 第 1 步: 发起 I2C 总线启动信号 */
17     i2c_Start();
18
19     /* 第 2 步: 发起控制字节, 高 7bit 是地址, bit0 是读写控制位, 0 表示写, 1 表示读 */
20     i2c_SendByte(ClientAddr | I2C_DIR_RD); /* 此处是读指令 */
21
22     /* 第 3 步: 等待 ACK */
23     if (i2c_WaitAck() != 0) {
24         goto cmd_fail; /* 器件无应答 */
25     }
26
27     while (NumByteToRead) {
28         if (NumByteToRead == 1) {
```

```
29         i2c_NAck(); /* 最后 1 个字节读完后, CPU 产生 NACK 信号 (驱动 SDA = 1) */
30     /*
31         /* 发送 I2C 总线停止信号 */
32         i2c_Stop();
33     }
34
35     *pBuffer = i2c_ReadByte();
36
37     /* 读指针自增 */
38     pBuffer++;
39
40     /* 计数器自减 */
41     NumByteToRead--;
42
43     i2c_Ack(); /* 中间字节读完后, CPU 产生 ACK 信号 (驱动 SDA = 0) */
44 }
45
46 /* 发送 I2C 总线停止信号 */
47 i2c_Stop();
48 return 0; /* 执行成功 */
49
50 cmd_fail: /* 命令执行失败后, 切记发送停止信号, 避免影响 I2C 总线上其他设备 */
51 /* 发送 I2C 总线停止信号 */
52 i2c_Stop();
53 return 1;
54 }
55
56 /**
57  * @brief 使用 IIC 写入数据
58  * @param
59  * @arg ClientAddr: 从设备地址
60  * @arg pBuffer: 缓冲区指针
61  * @arg NumByteToWrite: 写的字节数
62  * @retval 无
63  */
64 uint32_t I2C_WriteBytes(uint8_t ClientAddr, uint8_t* pBuffer, uint8_t NumByteToWrite)
65 {
66     uint16_t m;
67
68     /* 第 0 步: 发停止信号, 启动内部写操作 */
69     i2c_Stop();
70
71     /* 通过检查器件应答的方式, 判断内部写操作是否完成, 一般小于 10ms
72     CLK 频率为 200KHz 时, 查询次数为 30 次左右
73     */
74     for (m = 0; m < 1000; m++) {
75         /* 第 1 步: 发起 I2C 总线启动信号 */
76         i2c_Start();
77
78         /* 第 2 步: 发起控制字节, 高 7bit 是地址, bit0 是读写控制位, 0 表示写, 1 表示读 */
79         i2c_SendByte(ClientAddr | I2C_DIR_WR); /* 此处是写指令 */
80
81         /* 第 3 步: 发送一个时钟, 判断器件是否正确应答 */
82         if (i2c_WaitAck() == 0) {
83             break;
84         }
85     }
86     if (m == 1000) {
87         goto cmd_fail; /* EEPROM 器件写超时 */
88     }
89
90     while (NumByteToWrite-- > 0) {
```

```
91      /* 第 4 步：开始写入数据 */
92      i2c_SendByte(*pBuffer);
93
94      /* 第 5 步：检查 ACK */
95      if (i2c_WaitAck() != 0) {
96          goto cmd_fail; /* 器件无应答 */
97      }
98
99      pBuffer++; /* 地址增 1 */
100  }
101
102  /* 命令执行成功，发送 I2C 总线停止信号 */
103  i2c_Stop();
104  return 0;
105
106 cmd_fail: /* 命令执行失败后，切记发送停止信号，避免影响 I2C 总线上其他设备 */
107  /* 发送 I2C 总线停止信号 */
108  i2c_Stop();
109  return 1;
110 }
```

为了与后面的 gt5xx.c 文件的读写接口兼容，这里的读写函数都是很纯粹的 I2C 通讯过程，即读函数只有读过程，不包含发送寄存器地址的过程，而写函数也是只有写过程，没有包含寄存器的地址，大家可以对比一下它们与前面 EEPROM 实验中的差别。这两个函数都只包含从 I2C 的设备地址、缓冲区指针以及数据量。

Linux 的 I2C 驱动接口

使用前面的基本读写函数，主要是为了对接原“gt5xx.c”驱动里使用的 Linux I2C 接口函数 I2C_Transfer，实现了这个函数后，移植时就可以减少“gt5xx.c”文件的修改量。I2C_Transfer 函数见代码清单 30-8。

代码清单 30-8 Linux 的 I2C 驱动接口(gt5xx.c 文件)

```
1
2 /* 表示读数据 */
3 #define I2C_M_RD          0x0001
4 /*
5 * 存储 I2C 通讯的信息
6 * @addr: 从设备的 I2C 设备地址
7 * @flags: 控制标志
8 * @len: 读写数据的长度
9 * @buf: 存储读写数据的指针
10 */
11 struct i2c_msg
12 {
13     uint8_t addr; /*从设备的 I2C 设备地址 */
14     uint16_t flags; /*控制标志*/
15     uint16_t len; /*读写数据的长度 */
16     uint8_t *buf; /*存储读写数据的指针 */
17 };
18
19 /**
20 * @brief 使用 IIC 进行数据传输
21 * @param
22 * @arg i2c_msg:数据传输结构体
23 * @arg num:数据传输结构体的个数
24 * @retval 正常完成的传输结构个数，若不正常，返回 0xff
25 */
26 static int I2C_Transfer( struct i2c_msg *msgs,int num)
27 {
```

```
28     int im = 0;
29     int ret = 0;
30     //输出调试信息, 可忽略
31     GTP_DEBUG_FUNC();
32
33     for (im = 0; ret == 0 && im != num; im++)
34     {
35         //根据 flag 判断是读数据还是写数据
36         if ((msgs[im].flags & I2C_M_RD))
37         {
38             //IIC 读取数据
39             ret = I2C_ReadBytes(msgs[im].addr, msgs[im].buf, msgs[im].len);
40         }
41         else
42         {
43             //IIC 写入数据
44             ret = I2C_WriteBytes(msgs[im].addr, msgs[im].buf, msgs[im].len);
45         }
46     }
47
48     if (ret)
49         return ret;
50
51     return im; //正常完成的传输结构个数
52 }
```

I2C_Transfer 的主要输入参数是 i2c_msg 结构体的指针以及要传输多少个这样的结构体。i2c_msg 结构体包含以下几个成员:

(1) addr

这是从机的 I2C 设备地址, 通讯时无论是读方向还是写方向, 给这个成员赋值为写地址即可(本实验中为 0xBA)。

(2) flags

这个成员存储了控制标志, 它用于指示本 i2c_msg 结构体要求以什么方式来传输。在原 Linux 驱动中有很多控制方式, 在我们这个工程中, 只支持读或写控制标志, flags 被赋值为 I2C_M_RD 宏的时候表示读方向, 其余值表示写方向。

(3) len

本成员存储了要读写的数据长度。

(4) buf

本成员存储了指向读写数据缓冲区的指针。

利用这个结构体, 我们再来看 I2C_Transfer 函数做了什么工作。

(1) 输入参数中可能包含有多个要传输的 i2c_msg 结构体, 利用 for 循环把这些结构体一个个地传输出去;

(2) 传输的时候根据 i2c_msg 结构体中的 flags 标志, 确定应该调用 I2C 读函数还是写函数, 这些函数即前面定义的 I2C 基本读写函数。调用这些函数的时候, 以 i2c_msg 结构体的成员作为参数。

I2C 复合读写函数

理解了 I2C_Transfer 函数的代码, 我们发现它还是什么都没做, 只是对 I2C 基本读写函数封装了比较特别的调用形式而已, 而我们知道 GT5688 触控芯片都有很多不同的寄存器, 如果我们仅用上面的函数, 如何向特定寄存器写入参数或读取特定寄存器的内容呢?

这就需要再利用 I2C_Transfer 函数编写具有 I2C 通讯复合时序的读写函数了。Linux 驱动进行这样的封装是为了让它的核心层与具体设备独立开来，对于这个巨型系统，这样写代码是很有必要的，上述的 I2C_Transfer 函数属于 Linux 内部的驱动层，它对外提供接口，而像 GT5688、EEPROM 等使用 I2C 的设备，都利用这个接口编写自己具体的驱动文件，GT5688 的这些 I2C 复合读写函数见代码清单 30-9。

代码清单 30-9 I2C 复合读写函数 (gt9xx.c 文件)

```
1 //寄存器地址的长度
2 #define GTP_ADDR_LENGTH 2
3
4
5 /**
6  * @brief 从 IIC 设备中读取数据
7  * @param
8  * @arg client_addr:设备地址
9  * @arg buf[0~1]: 读取数据寄存器的起始地址
10 * @arg buf[2~len-1]: 存储读出来数据的缓冲 buffer
11 * @arg len: GTP_ADDR_LENGTH + read bytes count (寄存器地址长度+读取的数据字节数)
12 * @retval i2c_msgs 传输结构体的个数, 2 为成功, 其它为失败
13 */
14 static int32_t GTP_I2C_Read(uint8_t client_addr, uint8_t *buf, int32_t len)
15 {
16     struct i2c_msg msgs[2];
17     int32_t ret=-1;
18     int32_t retries = 0;
19
20     GTP_DEBUG_FUNC();
21     /*一个读数据的过程可以分为两个传输过程:
22     * 1. IIC 写入 要读取的寄存器地址
23     * 2. IIC 读取 数据
24     */
25
26     msgs[0].flags = !I2C_M_RD; //写入
27     msgs[0].addr = client_addr; //IIC 设备地址
28     msgs[0].len = GTP_ADDR_LENGTH; //寄存器地址为 2 字节 (即写入两字节的数据)
29     msgs[0].buf = &buf[0]; //buf[0~1] 存储的是要读取的寄存器地址
30
31     msgs[1].flags = I2C_M_RD; //读取
32     msgs[1].addr = client_addr; //IIC 设备地址
33     msgs[1].len = len - GTP_ADDR_LENGTH; //要读取的数据长度
34     msgs[1].buf = &buf[GTP_ADDR_LENGTH]; //buf[GTP_ADDR_LENGTH] 之后的缓冲区存储读出的数据
35
36     while (retries < 5) {
37         ret = I2C_Transfer(msgs, 2); //调用 IIC 数据传输过程函数, 有 2 个传输过程
38         if (ret == 2) break;
39         retries++;
40     }
41     if ((retries >= 5)) {
42         GTP_ERROR("I2C Read: 0x%04X, %d bytes failed, errcode: %d! Process reset.",
43                 (((uint16_t)(buf[0] << 8) | buf[1]), len-2, ret);
44     }
45     return ret;
46 }
47
48
49
50 /**
51  * @brief 向 IIC 设备写入数据
52  * @param
```

```
53 *   @arg client_addr:设备地址
54 *   @arg buf[0~1]: 要写入的数据寄存器的起始地址
55 *   @arg buf[2~len-1]: 要写入的数据
56 * @arg len:      GTP_ADDR_LENGTH + write bytes count (寄存器地址长度+写入的数据字节数)
57 * @retval  i2c_msgs 传输结构体的个数, 1 为成功, 其它为失败
58 */
59 static int32_t GTP_I2C_Write(uint8_t client_addr,uint8_t *buf,int32_t len)
60 {
61     struct i2c_msg msg;
62     int32_t ret = -1;
63     int32_t retries = 0;
64
65     GTP_DEBUG_FUNC();
66     /*一个写数据的过程只需要一个传输过程:
67      * 1. IIC 连续 写入 数据寄存器地址及数据
68      */
69     msg.flags = !I2C_M_RD;          //写入
70     msg.addr = client_addr;         //从设备地址
71     msg.len = len;                  //长度直接等于 (寄存器地址长度+写入的数据字节数)
72     msg.buf = buf;                  //直接连续写入缓冲区中的数据 (包括了寄存器地址)
73
74     while (retries < 5) {
75         ret = I2C_Transfer(&msg, 1); //调用 IIC 数据传输过程函数, 1 个传输过程
76         if (ret == 1)break;
77         retries++;
78     }
79     if ((retries >= 5)) {
80
81         GTP_ERROR("I2C Write: 0x%04X, %d bytes failed, errcode: %d! Process reset.",
82                 (((uint16_t)(buf[0] << 8)) | buf[1]) , len-2, ret);
83
84     }
85     return ret;
86 }
```

可以看到,复合读写函数都包含有 client_addr、buf 及 len 输入参数,其中 client_addr 表示 I2C 的设备地址, buf 存储了要读写的寄存器地址及数据, len 表示 buf 的长度。在函数的内部处理中,复合读写过程被分解成两个基本的读写过程,输入参数被转化存储到 i2c_msg 结构体中,每个基本读写过程使用一个 i2c_msg 结构体来表示,见表 30-2 和表 30-3。

表 30-2 复合读过程的步骤分解

复合读过程的步骤分解	说明
传输寄存器地址	这相当于一个 I2C 的基本写过程,写入一个 2 字节长度的寄存器地址, buf 指针的前两个字节内容被解释为寄存器地址。
从寄存器读取内容	这是一个 I2C 的基本读过程,读取到的数据存储到 buf 指针的第 3 个地址开始的空间中。

表 30-3 复合写过程的步骤分解

复合写过程的步骤分解	说明
传输寄存器地址	这相当于一个 I2C 的基本写过程,写入一个 2 字节长度的寄存器地址, buf 指针的前两个字节内容被解释为寄存器地址。
向寄存器写入内容	这也是一个 I2C 的基本写过程,写入的数据为 buf 指针的第 3 个地址开始的内容。

复合过程的分解主要是针对寄存器地址传输和实际数据传输来划分的，调用这两个复合读写过程的时候，我们需要注意 buf 的前两个字节为寄存器地址，且 len 的长度为 buf 的整体长度。

读取触控芯片的产品 ID 及版本号

利用上述复合读写函数，我们就可以使用 I2C 控制触控芯片了，首先是最简单的读取版本函数，见代码清单 30-10。

代码清单 30-10 读取触控芯片的产品 ID 及版本号（gt5xx.c 文件）

```
1
2 /*****
3 Function:
4   Read chip version.
5 Input:
6   client: i2c device
7   version: buffer to keep ic firmware version
8 Output:
9   read operation return.
10    2: succeed, otherwise: failed
11 *****/
12 int32_t GTP_Read_Version(void)
13 {
14     int32_t ret = -1;
15     uint8_t buf[8] = {GTP_REG_VERSION >> 8, GTP_REG_VERSION & 0xff}; //寄存器地址
16
17     GTP_DEBUG_FUNC();
18
19     ret = GTP_I2C_Read(GTP_ADDRESS, buf, sizeof(buf));
20     if (ret < 0) {
21         GTP_ERROR("GTP read version failed");
22         return ret;
23     }
24     if (buf[2] == '5') {
25         GTP_INFO("IC1 Version: %c%c%c%c_%02x%02x",
26                 buf[2], buf[3], buf[4], buf[5], buf[7], buf[6]);
27
28         //GT5688 芯片
29         if (buf[2] == '5' && buf[3] == '6' && buf[4] == '8' && buf[5] == '8')
30             touchIC = GT5688;
31     }
32
33     return ret;
34 }
```

这个函数定义了一个 8 字节的 buf 数组，并且向它的第 0 和第 1 个元素写入产品 ID 寄存器的地址，然后调用复合读取函数，即可从芯片中读取这些寄存器的信息，结果使用宏 GTP_INFO 输出。

向触控芯片写入配置参数

万事俱备，现在我们可以使用 I2C 向触摸芯片写入寄存器配置了，见代码清单 30-11。

代码清单 30-11 初始化并向触控芯片写入配置参数（gt5xx.c 文件）

```
1 // 4.5 寸屏 GT5688 驱动配置
2 const uint8_t CTP_CFG_GT5688[] = {
3     0x96, 0xE0, 0x01, 0x56, 0x03, 0x05, 0x35, 0x00, 0x01, 0x00,
4     0x00, 0x05, 0x50, 0x3C, 0x53, 0x11, 0x00, 0x00, 0x22, 0x22,
5     0x14, 0x18, 0x1A, 0x1D, 0x0A, 0x04, 0x00, 0x00, 0x00, 0x00,
6
7     /*省略部分寄存器内容*/
```

```
8      0x22,0x03,0x00,0x00,0x33,0x00,0x0F,0x00,0x00,0x00,
9      0x50,0x3C,0x50,0x00,0x00,0x00,0x1A,0x64,0x01
10 };
11
12 /*****
13 Function:
14     Initialize gtp.
15 Input:
16     ts: goodix private data
17 Output:
18     Executive outcomes.
19     0: succeed, otherwise: failed
20 *****/
21 int32_t GTP_Init_Panel(void)
22 {
23     int32_t ret = -1;
24
25     int32_t i = 0;
26     uint16_t check_sum = 0;
27     int32_t retry = 0;
28
29     const uint8_t* cfg_info;
30     uint8_t cfg_info_len ;
31     uint8_t* config;
32
33     uint8_t cfg_num = 0 ;    //需要配置的寄存器个数
34
35     GTP_DEBUG_FUNC();
36
37     config = (uint8_t *)malloc (GTP_CONFIG_MAX_LENGTH + GTP_ADDR_LENGTH);
38
39     config[0] = GTP_REG_CONFIG_DATA >> 8;
40     config[1] = GTP_REG_CONFIG_DATA & 0xff;
41
42     I2C_Touch_Init();
43
44     ret = GTP_I2C_Test();
45     if (ret < 0) {
46         GTP_ERROR("I2C communication ERROR!");
47         return ret;
48     }
49
50     //获取触摸 IC 的型号
51     GTP_Read_Version();
52
53     //根据 IC 的型号指向不同的配置
54     if (touchIC == GT5688) {
55         cfg_info = CTP_CFG_GT5688; //指向寄存器配置
56         cfg_info_len = CFG_GROUP_LEN(CTP_CFG_GT5688); //计算配置表的大小
57     }
58
59     memset(&config[GTP_ADDR_LENGTH], 0, GTP_CONFIG_MAX_LENGTH);
60     memcpy(&config[GTP_ADDR_LENGTH], cfg_info, cfg_info_len);
61
62     cfg_num = cfg_info_len;
63
64     GTP_DEBUG("cfg_info_len = %d ",cfg_info_len);
65     GTP_DEBUG("cfg_num = %d ",cfg_num);
66     GTP_DEBUG_ARRAY(config,6);
67
68     /*根据 LCD 的扫描方向设置分辨率*/
69     config[GTP_ADDR_LENGTH+1] = LCD_X_LENGTH & 0xFF;
70     config[GTP_ADDR_LENGTH+2] = LCD_X_LENGTH >> 8;
71     config[GTP_ADDR_LENGTH+3] = LCD_Y_LENGTH & 0xFF;
72     config[GTP_ADDR_LENGTH+4] = LCD_Y_LENGTH >> 8;
73
74     /*此处省略部分特殊配置...*/
```

```
75
76
77 //计算要写入 checksum 寄存器的值
78 check_sum = 0;
79
80 for (i = GTP_ADDR_LENGTH; i < (cfg_num+GTP_ADDR_LENGTH -3); i += 2) {
81     check_sum += (config[i] << 8) + config[i + 1];
82 }
83
84 check_sum = 0 - check_sum;
85 GTP_DEBUG("Config checksum: 0x%04X", check_sum);
86 //更新 checksum
87 config[(cfg_num+GTP_ADDR_LENGTH -3)] = (check_sum >> 8) & 0xFF;
88 config[(cfg_num+GTP_ADDR_LENGTH -2)] = check_sum & 0xFF;
89 config[(cfg_num+GTP_ADDR_LENGTH -1)] = 0x01;
90
91 //写入配置信息
92 for (retry = 0; retry < 5; retry++) {
93     ret = GTP_I2C_Write(GTP_ADDRESS, config, cfg_num + GTP_ADDR_LENGTH+2);
94     if (ret > 0) {
95         break;
96     }
97 }
98 Delay(0xfffff); //延迟等待芯片更新
99
100 /*使能中断，这样才能检测触摸数据*/
101 I2C_GTP_IRQEnable();
102
103 GTP_Get_Info();
104
105 free(config);
106
107 return 0;
108 }
```

这段代码调用 I2C_Touch_Init 初始化了 STM32 的 I2C 外设，设定触控芯片的 I2C 设备地址，然后调用了 GTP_Read_Version 尝试获取触控芯片的版本号。接下来是函数的主体，它使用 GTP_I2C_Write 函数通过 I2C 把配置参数表 CTP_CFG_GT5688 写入到触控芯片的配置寄存器中，注意传输中包含有 checksum 寄存器的值。写入完参数后调用 I2C_GTP_IRQEnable 以使能 INT 引脚检测中断。

INT 中断服务函数

经过上面的函数初始化后，触摸屏就可以开始工作了，当触摸时，INT 引脚会产生触摸中断，会进入中断服务函数 GTP_IRQHandler，见代码清单 30-12。

代码清单 30-12 触摸屏的中断服务函数(stm32f4xx_it.c 文件)

```
1 //触摸屏中断服务函数
2 void GTP_IRQHandler(void)
3 {
4     //确保是否产生了 EXTI Line 中断
5     if (EXTI_GetITStatus(GTP_INT_EXTI_LINE) != RESET)
6     {
7         GTP_TouchProcess();
8         EXTI_ClearITPendingBit(GTP_INT_EXTI_LINE); //清除中断标志位
9     }
10 }
```

中断服务函数只是简单地调用了 GTP_TouchProcess 函数，它是读取触摸坐标的主体，而它又直接调用了函数 Goodix_TS_Work_Func。

读取坐标数据

Goodix_TS_Work_Func 函数的内容见代码清单 30-13。

代码清单 30-13 Goodix_TS_Work_Func 坐标读取函数

```
1  /**
2   * @brief 触屏处理函数，轮询或者在触摸中断调用
3   * @param 无
4   * @retval 无
5   */
6  static void Goodix_TS_Work_Func(void)
7  {
8      uint8_t end_cmd[3] = {GTP_READ_COOR_ADDR >> 8, GTP_READ_COOR_ADDR & 0xFF, 0};
9      uint8_t point_data[2 + 1 + 8 * GTP_MAX_TOUCH + 1] =
10         {GTP_READ_COOR_ADDR >> 8, GTP_READ_COOR_ADDR & 0xFF};
11      uint8_t touch_num = 0;
12      uint8_t finger = 0;
13      static uint16_t pre_touch = 0;
14      static uint8_t pre_id[GTP_MAX_TOUCH] = {0};
15
16      uint8_t client_addr=GTP_ADDRESS;
17      uint8_t* coor_data = NULL;
18      int32_t input_x = 0;
19      int32_t input_y = 0;
20      int32_t input_w = 0;
21      uint8_t id = 0;
22
23      int32_t i = 0;
24      int32_t ret = -1;
25
26      GTP_DEBUG_FUNC();
27
28      ret = GTP_I2C_Read(client_addr, point_data, 12); //10 字节寄存器加 2 字节地址
29      if (ret < 0) {
30          GTP_ERROR("I2C transfer error. errno:%d\n ", ret);
31          return;
32      }
33
34      finger = point_data[GTP_ADDR_LENGTH]; //状态寄存器数据
35
36      if (finger == 0x00) { //没有数据，退出
37          return;
38      }
39
40      if ((finger & 0x80) == 0) { //判断buffer status 位
41          goto exit_work_func; //坐标未就绪，数据无效
42      }
43
44      touch_num = finger & 0x0f; //坐标点数
45      if (touch_num > GTP_MAX_TOUCH) {
46          goto exit_work_func; //大于最大支持点数，错误退出
47      }
48
49      if (touch_num > 1) { //不止一个点
50          uint8_t buf[8 * GTP_MAX_TOUCH] = {(GTP_READ_COOR_ADDR + 10) >> 8,
51                                             (GTP_READ_COOR_ADDR + 10) & 0xff};
52
53          ret = GTP_I2C_Read(client_addr, buf, 2 + 8 * (touch_num - 1));
54          memcpy(&point_data[12], &buf[2], 8 * (touch_num - 1));
55          //复制其余点数的数据到 point_data
56
57          if (pre_touch > touch_num) { //pre_touch > touch_num, 表示有的点释放了
```



```

57         for (i = 0; i < pre_touch; i++) { //一个点一个点处理
58             uint8_t j;
59             for (j=0; j<touch_num; j++) {
60                 coor_data = &point_data[j * 8 + 3];
61                 id = coor_data[0] & 0x0F; //track id
62                 if (pre_id[i] == id)
63                     break;
64                 //遍历当前所有 id 都找不到 pre_id[i], 表示已释放
65                 if (j >= touch_num-1) {
66                     GTP_Touch_Up( pre_id[i]);
67                 }
68             }
69         }
70     }
71
72     if (touch_num) {
73         for (i = 0; i < touch_num; i++) { //一个点一个点处理
74             coor_data = &point_data[i * 8 + 3];
75
76             id = coor_data[0] & 0x0F; //track id
77             pre_id[i] = id;
78
79             input_x = coor_data[1] | (coor_data[2] << 8); //x 坐标
80             input_y = coor_data[3] | (coor_data[4] << 8); //y 坐标
81             input_w = coor_data[5] | (coor_data[6] << 8); //size
82
83             {
84
85                 /*根据液晶扫描模式更正 X/Y 起始方向, LCD_SCAN_MODE 是液晶驱动配置的全局变量*/
86                 switch (LCD_SCAN_MODE) {
87                     case 0:
88                     case 7:
89                         input_y = LCD_Y_LENGTH - input_y;
90                         break;
91
92                     case 2:
93                     case 3:
94                         input_x = LCD_X_LENGTH - input_x;
95                         input_y = LCD_Y_LENGTH - input_y;
96                         break;
97
98                     case 1:
99                     case 6:
100                        input_x = LCD_X_LENGTH - input_x;
101                        break;
102
103                     default:
104                         break;
105                 }
106
107                 GTP_Touch_Down( id, input_x, input_y, input_w); //数据处理
108             }
109         }
110     } else if (pre_touch) { //touch_num=0 且 pre_touch!=0
111         for (i=0; i<pre_touch; i++) {
112             GTP_Touch_Up(pre_id[i]);
113         }
114     }
115
116     pre_touch = touch_num;
117
118 exit_work_func: {
119     ret = GTP_I2C_Write(client_addr, end_cmd, 3);
120     if (ret < 0) {
121         GTP_INFO("I2C write end_cmd error!");
122     }
123 }

```

这个函数的内容比较长，它首先是读取了状态寄存器，获当前有多少个触点，然后根据触点数去读取各个点的数据，其中还有包含有 pre_touch 点的处理，pre_touch 保存了上一次的触点数据，利用这些数据和触点的 track id 号，可以确认同一条笔迹。这个读取过程完毕后，还对状态寄存器的 buffer status 位写 0，结束读取。在实际应用中，我们并不需要掌握这个 Goodix_TS_Work_Func 函数的所有细节，因为在这个函数中提供了两个坐标获取接口，我们只要在这两个接口中修改即可简单地得到坐标信息。

触点释放和触点按下的坐标接口

Goodix_TS_Work_Func 函数中获取到新的坐标数据时会调用触点释放和触点按下这两个函数，我们只要在这两个函数中添加自己的坐标处理过程即可，见代码清单 30-14。

代码清单 30-14 触点释放和触点按下的坐标接口（gt5xx.c 文件）

```
1
2 /**
3  * @brief 用于处理或报告触屏检测到按下
4  * @param
5  *   @arg id: 触摸顺序 trackID
6  *   @arg x: 触摸的 x 坐标
7  *   @arg y: 触摸的 y 坐标
8  *   @arg w: 触摸的 大小
9  * @retval 无
10 */
11 /*用于记录连续触摸时(长按)的上一次触摸位置，负数值表示上一次无触摸按下*/
12 static int16_t pre_x[GTP_MAX_TOUCH] = {-1,-1,-1,-1,-1};
13 static int16_t pre_y[GTP_MAX_TOUCH] = {-1,-1,-1,-1,-1};
14
15 static void GTP_Touch_Down(int32_t id,int32_t x,int32_t y,int32_t w)
16 {
17     GTP_DEBUG_FUNC();
18
19     /*取 x、y 初始值大于屏幕像素值*/
20     GTP_DEBUG("ID:%d, X:%d, Y:%d, W:%d", id, x, y, w);
21
22     /* 处理触摸按钮，用于触摸画板 */
23     Touch_Button_Down(x,y);
24     /*处理描绘轨迹，用于触摸画板 */
25     Draw_Trail(pre_x[id],pre_y[id],x,y,&brush);
26
27     /******
28     /*在此处添加自己的触摸点按下时处理过程即可*/
29     /* (x,y) 即为最新的触摸点 *****/
30     /******
31
32     /*pre_x,pre_y 数组存储上一次触摸的位置，id 为轨迹编号(多点触控时有多轨迹)*/
33     pre_x[id] = x;
34     pre_y[id] = y;
35 }
36
37 }
38
39
40 /**
41  * @brief 用于处理或报告触屏释放
42  * @param 释放点的 id 号
43  * @retval 无
44  */
45 static void GTP_Touch_Up( int32_t id)
```

```
46 {  
47     /*处理触摸释放,用于触摸画板*/  
48     Touch_Button_Up(pre_x[id],pre_y[id]);  
49  
50     /******/  
51     /*在此处添加自己的触摸点释放时的处理过程即可*/  
52     /* pre_x[id],pre_y[id] 即为最新的释放点 *****/  
53     /******/  
54     /***id 为轨迹编号(多点触控时有多轨迹)*****/  
55  
56  
57     /*触笔释放,把pre xy 重置为负*/  
58     pre_x[id] = -1;  
59     pre_y[id] = -1;  
60  
61     GTP_DEBUG("Touch id[%2d] release!", id);  
62 }
```

以上是我们工程中对这两个接口的应用,我们把触摸画板的坐标处理过程直接放到接口里了,大家可参考我们的演示,在函数的注释部分,根据自己的应用编写坐标处理过程。

注意这两个坐标接口都还是在中断服务函数里调用的(中断服务函数调用 Goodix_TS_Work_Func 函数,该函数再调用这两个坐标接口),实际应用中可以先把这些坐标信息存储起来,等待到系统空闲的时候再处理,就可以减轻中断服务程序的负担了。

3. main 函数

完成了触摸屏的驱动,就可以应用了,以下我们来看工程的主体 main 函数,见代码清单 25-14。

代码清单 30-15 main 函数

```
1 /**  
2  * @brief 主函数  
3  * @param 无  
4  * @retval 无  
5  */  
6 int main ( void )  
7 {  
8     LED_GPIO_Config();  
9     ILI9806G_Init ();           //LCD 初始化  
10  
11     Debug_USART_Config();  
12  
13     //其中 0、3、5、6 模式适合从左至右显示文字,  
14     //不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果  
15     //其中 6 模式为大部分液晶例程的默认显示方向  
16     ILI9806G_GramScan ( 3 );  
17  
18     /* 设定好液晶扫描方向后,再初始化触摸屏,  
19     触摸屏会根据液晶的扫描方向输出匹配的触摸坐标 */  
20     /* 每次修改液晶扫描方向后,  
21     应重新调用一次 GTP_Init_Panel 函数更新触摸配置 */  
22     GTP_Init_Panel();  
23  
24     printf("\r\n ***** 触摸画板程序 ***** \r\n");  
25     printf("\r\n 若汉字显示不正常,请阅读工程中的 readme.txt 文件说明,  
26     根据要求给 FLASH 重刷字模数据\r\n");  
27 }
```

```
28    //绘制触摸画板界面
29    Palette_Init(LCD_SCAN_MODE);
30
31    while ( 1 ) {
32    }
33 }
```

main 函数初始化触摸屏、液晶屏后，调用了 Palette_Init 函数初始化了触摸画板应用，关于触摸画板应用的内容在“palette.c”及“palette.h”文件中，这些都是与 STM32 无关上层应用，感兴趣的读者可在工程中阅读，本教程就不讲解这些内容了。

下载验证

编译程序下载到实验板，并上电复位，液晶屏会显示出触摸画板的界面，点击屏幕可以在该界面画出简单的图形。